PROJET FINAL · POEI LLS EXPLEO · AJC FORMATION

EmbedGate

Une passerelle de supervision embarquée : de la puce capteur au tableau de bord de l'opérateur

Groupe [N]

[Prénom NOM] · [Prénom NOM] · [Prénom NOM]

Soutenance · lundi 26 octobre 2026

Dépôt : tag v1.0

L'ÉQUIPE

Qui a construit EmbedGate

[Prénom NOM]

Avant la formation
[études, métier, domaine]

Dans EmbedGate
[ex. : le pilote C et la carte ARM]

Mes points forts
[une ou deux compétences]

Je vise
[poste ou domaine]

[Prénom NOM]

Avant la formation
[études, métier, domaine]

Dans EmbedGate
[ex. : la passerelle C++]

Mes points forts
[une ou deux compétences]

Je vise
[poste ou domaine]

[Prénom NOM]

Avant la formation
[études, métier, domaine]

Dans EmbedGate
[ex. : le superviseur Python]

Mes points forts
[une ou deux compétences]

Je vise
[poste ou domaine]

LE BESOIN DU CLIENT

3 h du matin : la chambre froide se réchauffe

4 °C

Tout va bien. Le tableau de bord est vert.

6 °C…

Un joint de porte cède. La température monte doucement.

8,5 °C

Seuil de 8 °C dépassé : **alerte immédiate**, ligne rouge, trace au journal.

Retour

Porte réparée : retour à la normale signalé, une seule fois.

Sans surveillance, la panne n’est vue qu’à l’ouverture : la marchandise est perdue.

EmbedGate **surveille** les capteurs en continu, **alerte** dès qu’une valeur sort de sa plage et laisse l’opérateur **agir à distance**.

LA SOLUTION

Une chaîne en cinq maillons, de la puce à l'écran

fournie

Puce capteur simulée

Registres, interruptions, scénarios de panne

→

C

Pilote

Lit les registres (MMIO), sert les interruptions

→

C++17

Passerelle

Threads, file, alarmes, journal, commandes

↔

MQTT

Broker

Mosquitto : publication et abonnement

↔

Python

Superviseur

Tableau de bord, envoi des commandes

Carte ARM Le pilote et la passerelle sont compilés en croisé et tournent sur une carte ARM émulée (QEMU, image Buildroot). Le superviseur, sur le PC, ne voit aucune différence.

Contrat imposé et commun aux six groupes : registres de la puce, messages MQTT, format du journal. À l'intérieur, les choix sont les nôtres : [un choix marquant, en une ligne].

LE CURSUS DANS NOTRE PROJET · 64 JOURS, DU 15 JUILLET AU 26 OCTOBRE

Chaque module a laissé sa trace dans EmbedGate

Modules	Où, dans EmbedGate	Notre preuve
Commandes Linux, Shell, Administration et Architecture Linux	demo.sh, lecture du journal, signaux et arrêt propre	[ex. : demo.sh]
Git	branches, relectures, tags v0.1 à v1.0	[ex. : git log]
Programmation C, Projet C	pilote : registres, volatile, bits, interruptions, rappels	[ex. : pilote_demo]
Temps réel	interruptions simulées, file entre threads, cadence tenue	[ex. : rafale]
C++, Modern C++ 11, Projet C++	passerelle : RAII, threads, Strategy, Observer	[ex. : tests C++]
Python, Dev. open source	superviseur, MQTT, build CMake	[ex. : pytest]
Tests unitaires	tests C, C++ et Python ; moteur d'alarmes en TDD	[ex. : ctest]
Debug, Trace32	Valgrind, ThreadSanitizer, gdb et gdbserver	[ex. : Valgrind]
Linux embarqué	compilation croisée, Buildroot, QEMU, démarrage au boot	[ex. : carte ARM]
Algorithmie, Consultant	machine à états de l'alarme ; journal de bord, soutenance	[ex. : JOURNAL.md]

DÉMONSTRATION EN DIRECT · [6] MIN

Ce que vous allez voir

#	Nous faisons	Vous devez voir
1	Lancer la puce, la passerelle, le superviseur	Mesures à jour, état « en ligne »
2	Rejouer « chambre froide »	Alarme HAUT à 8,5 °C, ligne rouge, une seule fois
3	Laisser la température redescendre	Retour à la normale signalé
4	Envoyer un calibrage de −1,5 °C	Accusé « ok », valeur corrigée à l'écran
5	Couper la passerelle	État « hors ligne » au tableau de bord
6	Même chaîne sur la carte ARM émulée	Mesures venues de la carte, sur le PC
7	[extension]	[ce qu'elle montre]

Une seule commande lance l'essentiel : scripts/demo.sh → DEMO OK.

Le pilote parle à la puce comme à du vrai matériel

- Registres projetés en mémoire (mmap), lus par pointeur volatile
- Masques et champs de bits : on modifie un bit sans toucher aux autres
- Interruptions simulées (timerfd) : la routine note et acquitte, le traitement se fait ailleurs
- Pannes de capteur détectées et signalées, sans arrêter les autres canaux
- Une API C simple : la passerelle s'abonne aux mesures par fonction de rappel

Notre choix

[Un choix et sa raison. Exemple : pourquoi la routine d'interruption ne fait presque rien.]

La preuve

pilote_demo sur PC et sur ARM · [N] tests C verts

La passerelle collecte, décide et prévient



- **RAII** : chaque ressource est libérée par son destructeur, aucun delete à la main
- **Hystérésis** : une alerte par anomalie, pas une rafale
- **Strategy** : une règle d’alarme interchangeable par canal
- **Observer** : journal et MQTT sont prévenus du même événement
- **Arrêt propre** sur SIGTERM : la file est vidée, rien n’est perdu

Notre choix

[Exemple : pourquoi deux threads et une file plutôt qu’un seul thread.]

La preuve

Moteur d’alarmes écrit en TDD · rafale : [N] mesures, 0 perte

L’opérateur voit tout et agit à distance

Message	Conservé ?	Pourquoi
mesure	non	la suivante arrive aussitôt
alarme, panne	oui	un écran ouvert après coup voit l’état
etat	oui	« hors ligne » même après un plantage (testament)
cmd → cmd/ack	non	chaque ordre reçoit sa réponse, par son numéro

Commandes : calibrage, remise à zéro, auto-test, arrêt et reprise de l’acquisition. Une commande invalide est refusée, jamais fatale.

Notre choix

[Exemple : comment le tableau de bord reste lisible avec quatre canaux.]

La preuve

[N] tests pytest verts · broker coupé puis relancé : reprise seule

MAILLON 4 · LINUX EMBARQUÉ

Le même logiciel, sur un petit boîtier ARM

1

Compiler en croisé

Sur le PC, pour ARM : CMake et la chaîne arm-linux-gnueabihf

2

Démarrer la carte

Image Buildroot, carte ARM émulée par QEMU

3

Déployer

Passerelle lancée seule au démarrage (script d'init)

4

Déboguer à distance

gdbserver sur la carte, gdb sur le PC

```
$ file build-arm/embedgate-gw
ELF 32-bit LSB executable, ARM
$ mosquitto_sub -t 'embedgate/[gpN]/etat'
{"etat":"en_ligne"}
```

La preuve

« chambre froide » sur la carte : mêmes alarmes que sur le PC

[Capture : point d'arrêt atteint dans la routine d'interruption]

QUALITÉ · TESTS ET DÉBOGAGE

Ce que nous avons mesuré, pas seulement affirmé

[N]
tests automatiques, C, C++ et Python, tous verts

0
fuite et accès invalide sous Valgrind, commandes comprises

0
conflit entre threads sous ThreadSanitizer

[N]
mesures en rafale, aucune perdue

0
avertissement de compilation, sur PC et pour ARM

[x] %
de couverture du moteur d'alarmes (gcov, si mesurée)

Démarche : les tests du moteur d'alarmes ont été écrits **avant** lui (TDD). Un bug marquant trouvé par un outil : [lequel, avec quel outil, comment corrigé].

MÉTHODE · GIT ET ORGANISATION

Dix jours, six phases, un logiciel qui marche à chaque étape

v0.1	v0.2	v0.3	v0.4	v0.5	v1.0
pilote C	chaîne complète	alarmes	commandes	carte ARM	livraison

Comment nous avons travaillé

- Une branche par fonctionnalité, main compile toujours
- Un porteur et un relecteur par brique, rôles échangés à chaque phase
- Un journal de bord quotidien : décisions, blocages, solutions

Qui a porté quoi

Membre	A porté	Commits
[Prénom]	[briques]	[N]
[Prénom]	[briques]	[N]
[Prénom]	[briques]	[N]

DIFFICULTÉS · TIRÉES DE NOTRE JOURNAL DE BORD

Trois problèmes réels, et comment nous les avons résolus

Le problème	Comment nous l'avons trouvé	La solution
[symptôme observé]	[outil, test ou démarche]	[correction et leçon]
[symptôme observé]	[outil, test ou démarche]	[correction et leçon]
[symptôme observé]	[outil, test ou démarche]	[correction et leçon]

Exemples du type de problème attendu : une alarme qui se répète en boucle autour du seuil ; une fuite mémoire signalée par Valgrind ; un conflit entre threads détecté par ThreadSanitizer ; la passerelle qui ne s'arrête pas proprement ; un binaire ARM qui ne démarre pas sur la carte.

AU-DELÀ DU SOCLE

Ce que nous avons ajouté, et ce qui manque encore

Extensions réalisées

[E?] [nom : ce qu'elle apporte à l'opérateur, en une ligne]

[E?] [nom : ce qu'elle apporte, en une ligne]

Au choix : E1 bus CAN · E2 tableau de bord web · E3 historique SQLite et CSV · E4 alarme temporisée · E5 CI, cppcheck, gcov, Doxygen · E6 paquet Debian et systemd · deux au moins pour le trinôme

Limites connues

[Ce qui ne marche pas ou pas complètement, honnêtement]

Avec plus de temps

[Ce que nous ferions ensuite, pour un vrai client]

BILAN

Ce que chacun de nous retient

[Prénom]

« [Une phrase : ce que j'ai appris ou ce que je sais faire maintenant.] »

[Prénom]

« [Une phrase : ce que j'ai appris ou ce que je sais faire maintenant.] »

[Prénom]

« [Une phrase : ce que j'ai appris ou ce que je sais faire maintenant.] »

Merci. Vos questions ?

Dépôt : [adresse GitLab]
Tag : v1.0